
DTO

Data Transfer Object



Prof. Dr. João Choma

UEM

github.com/JoaoChoma

Motivação



Ao final da aula, você deverá ser capaz de:

- Explicar por que uma API não deve expor entidades diretamente.
- Diferenciar DTOs de entrada, saída, resumo, detalhe, erro e integração.
- Relacionar DTOs à separação de responsabilidades.
- Implementar DTOs com Controller, Service e Mapper.



O restaurante não entrega a cozinha inteira

- A cozinha possui estoque, receitas, custos e equipamentos.
- O cliente recebe somente o pedido pronto.
- O DTO funciona como uma **caixa de entrega**.

Ideia central

Transportar dados sem expor toda a estrutura interna.





Sem DTO	Com DTO
Controller devolve Entity	Controller usa objetos próprios da API
Cliente vê campos internos	Entity permanece protegida
Mudança na Entity quebra a API	Contrato pode evoluir separadamente
Responsabilidades se misturam	Cada classe tem uma intenção clara

Regra de projeto

Entidade de persistência $\neq$ contrato público da API.



Data Transfer Object

Objeto criado para transportar dados entre camadas, processos ou sistemas.

- Não é um dos 23 padrões GoF.
- É um padrão arquitetural ou de aplicação empresarial.
- Também aparece na literatura como *Transfer Object*.
- É comum em APIs REST, microsserviços e aplicações em camadas.

Sua função é definir uma fronteira e um contrato de dados.



Por que o DTO existe?

- Proteger senha, token e flags internas.
- Controlar entrada e saída da API.
- Desacoplar banco, domínio e cliente.
- Facilitar versionamento do contrato.
- Enviar somente os dados necessários.





```
@Entity
public class Usuario {
    @Id
    private Long id;
    private String nome;
    private String email;
    private String senhaHash;
    private Boolean administrador;
    private LocalDateTime criadoEm;
}

@GetMapping("/usuarios/{id}")
public Usuario buscar(@PathVariable Long id) {
    return usuarioService.buscarEntidade(id);
}
```




O que pode dar errado?

- `senhaHash` pode aparecer na resposta.
- `administrador` expõe uma regra interna.
- Mudanças no banco alteram o contrato da API.
- Associações JPA podem gerar respostas enormes ou ciclos no JSON.
- A Entity passa a servir dois mundos: banco e internet.

Cuidado

A entidade não deve virar contrato público por acidente.

Contratos e responsabilidades



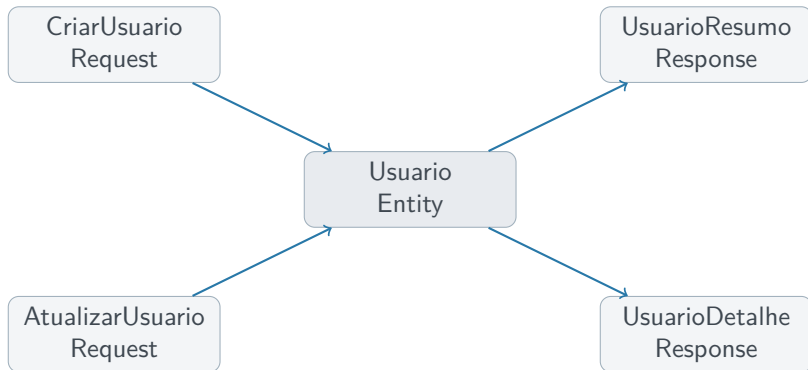
Componente	Responsabilidade
Entity	Persistência e estado interno
Request DTO	Dados que entram na API
Response DTO	Dados que saem da API
Mapper	Conversão entre Entity e DTOs
Service	Regras e decisões do caso de uso

O DTO carrega dados; ele não executa a regra de negócio.



- 1 Cliente envia dados.
- 2 Request DTO representa a entrada.
- 3 Controller delega ao Service.
- 4 Service trabalha com o domínio.
- 5 Mapper produz o Response DTO.
- 6 Cliente recebe o contrato previsto.





Cada caso de uso recebe ou devolve exatamente os dados de que precisa.

Tipos de DTO



Tipos de DTO

Request entrada em POST ou PUT.

Create dados necessários para criação.

Update/Patch dados permitidos na alteração.

Response resposta entregue ao cliente.

Summary poucos campos para listagem.

Detail visão detalhada do recurso.

Error formato previsível de falha.

Integration contrato de outro sistema.





```
public record CriarUsuarioRequest(  
    @NotBlank String nome,  
    @Email String email,  
    @Size(min = 8) String senha  
) { }
```

- Representa o corpo da requisição.
- Declara validações do contrato de entrada.
- Impede que o Controller receba uma Entity.
- Não executa regra de negócio.



```
public record UsuarioResponse(  
    Long id,  
    String nome,  
    String email  
) { }
```

Decisão consciente

Ficaram de fora: senhaHash, administrador, datas internas e associações pesadas.

Response DTO é contrato de saída, não um espelho automático da Entity.



```
// Listagem: resposta compacta
public record UsuarioResumoResponse(
    Long id,
    String nome
) { }

// Tela de detalhes: mais contexto
public record UsuarioDetalheResponse(
    Long id,
    String nome,
    String email,
    LocalDateTime criadoEm
) { }
```

Listagem pede poucos dados; detalhe pode exigir mais informação.



```
public record ErrorResponse(  
    String codigo,  
    String mensagem,  
    String campo  
) { }
```

```
{  
  "codigo": "VALIDACAO",  
  "mensagem": "E-mail invalido",  
  "campo": "email"  
}
```

- O cliente sabe interpretar a falha.
- Mensagens técnicas não vazam.
- Testes da API ficam mais simples.



```
// Contrato da API externa
public record ViaCepResponseDTO(
    String logradouro,
    String localidade,
    String uf
) { }

// Contrato da nossa API
public record EnderecoResponse(
    String rua,
    String cidade,
    String estado
) { }
```

Fronteira

O DTO externo não deve dominar o modelo interno da aplicação.

Implementação em Java



```
@Entity
public class Usuario {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;
    private String email;
    private String senhaHash;
    private Boolean administrador;

    protected Usuario() { }

    public Usuario(String nome, String email, String senhaHash) {
        this.nome = nome;
        this.email = email;
        this.senhaHash = senhaHash;
        this.administrador = false;
    }
}
```



```
public record CriarUsuarioRequest(  
    @NotBlank String nome,  
    @Email String email,  
    @Size(min = 8) String senha  
) { }  
  
public record UsuarioResponse(  
    Long id,  
    String nome,  
    String email  
) { }
```

A senha entra no Request, mas nunca volta no Response.



```
@Component
public class UsuarioMapper {
    public Usuario toEntity(CriarUsuarioRequest request,
                           String senhaHash) {
        return new Usuario(request.nome(),
                           request.email(), senhaHash);
    }

    public UsuarioResponse toResponse(Usuario usuario) {
        return new UsuarioResponse(
            usuario.getId(),
            usuario.getNome(),
            usuario.getEmail()
        );
    }
}
```




Responsabilidade do Mapper

- Traduzir formatos entre a fronteira e o domínio.
- Manter o Controller simples.
- Manter o Service focado no caso de uso.
- Evitar conversões duplicadas pela aplicação.

Limite

O Mapper conhece os formatos, mas não decide regras de negócio.



```
@Service
public class UsuarioService {
    private final UsuarioRepository repository;
    private final UsuarioMapper mapper;
    private final PasswordEncoder encoder;

    public UsuarioResponse criar(CriarUsuarioRequest request) {
        String hash = encoder.encode(request.senha());
        Usuario usuario = mapper.toEntity(request, hash);
        Usuario salvo = repository.save(usuario);
        return mapper.toResponse(salvo);
    }
}
```

A regra e a orquestração ficam no Service; o DTO apenas atravessa a fronteira.

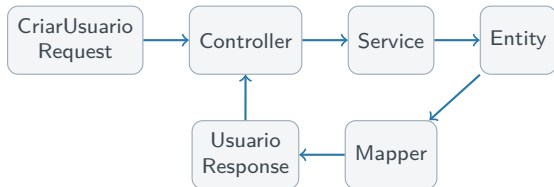


```
@RestController
@RequestMapping("/usuarios")
public class UsuarioController {
    private final UsuarioService service;

    public UsuarioController(UsuarioService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<UsuarioResponse> criar(
        @Valid @RequestBody CriarUsuarioRequest request) {

        UsuarioResponse response = service.criar(request);
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(response);
    }
}
```



- A Entity não aparece no contrato público.
- Controller fala HTTP; Service decide; Mapper converte.
- A API pode evoluir sem espelhar cada mudança do banco.

Decisões de projeto



Use quando

- A API não deve expor a Entity.
- Entrada e saída são diferentes.
- Existem dados sensíveis.
- O contrato exige validação.
- A resposta precisa ser customizada.

Evite exagero quando

- O sistema é muito simples e interno.
- O DTO apenas copia campos sem propósito.
- A conversão custa mais que o benefício.

DTO é uma decisão de fronteira, não uma cerimônia automática.



- DTO significa *Data Transfer Object*.
- É um padrão de transferência de dados, não um padrão GoF.
- Protege o domínio e estabiliza contratos.
- Request, Response, Summary, Detail, Error e Integration têm papéis distintos.
- Entity persiste, DTO transporta, Mapper converte e Service decide.

Frase-chave

Não entregue a cozinha; entregue a caixa certa.